La Faille

Stabiliser les promesses

Les premiers signaux arrivent correctement, mais Nyx veut éviter de transmettre des callbacks partout dans le système.

Elle vous montre un autre mécanisme :

> « Une promesse représente un résultat qui arrivera plus tard… ou une erreur. »

Vous allez utiliser :

- `Promise` ;
- `resolve` ;
- `reject` ;
- `then()` ;
- `catch()`.

Instructions

Complétez les fonctions du fichier `promesses.js`.

1. Créer une promesse résolue

Implémentez :

```
createSignal(message)
```

La fonction retourne une `Promise` immédiatement résolue avec `message`.

```
createSignal("NEXUS")
// Promise résolue avec "NEXUS"
```

2. Créer une promesse différée

Implémentez :

```
delayValue(value, delay)
```

Retournez une `Promise` qui se résout avec `value` après `delay` millisecondes.

Utilisez `setTimeout()` dans la promesse.

3. Valider un code temporel

Implémentez :

```
checkTemporalCode(code)
```

La fonction retourne une `Promise`.

- si `code === "NYX"`, la promesse est résolue avec `"Code accepté"` ;
- sinon elle est rejetée avec :

```
new Error("Code refusé")
```

4. Transformer une promesse avec `then`

Implémentez :

```
prepareMessage(message)
```

La fonction doit appeler `createSignal(message)` puis utiliser `.then()` pour retourner :

```
Signal : <message>
```

Exemple :

```
await prepareMessage("NEXUS")
// => "Signal : NEXUS"
```

5. Gérer un rejet avec `catch`

Implémentez :

```
safeTemporalCode(code)
```

Appelez `checkTemporalCode(code)` puis utilisez `.catch()`.

- si le code est valide, conservez `"Code accepté"` ;
- s'il est invalide, retournez `"Code refusé"`.

À rechercher si nécessaire

- `JavaScript Promise`
- `Promise resolve reject`
- `JavaScript then`
- `JavaScript catch`

▶ Aides